PR Wrangler: Demo Video (4 min)

Time	Section
0:00–0:30	Problem
0:30–1:30	Demo (the published output, no live run)
1:30–2:40	Architecture
2:40–3:20	Guardrails
3:20–4:05	Evals in LangSmith
4:05–4:15	Close

Before you hit record

- **Don't run `seed/reset.py`.** It deletes the published output you're about to show. Everything is live on GitHub right now (checked: queue issue #17 pinned, all 15 PRs labeled and commented).
- **Check Slack:** the digest and DMs are still in `#daily-pr-review`.
- **Keep `.env` off screen.** Rotate the GitHub, LangSmith and Slack keys pasted in chat earlier.
- **Open these tabs, in order:**
  1. GitHub: open PR list for `dhanush014/tiny-todo`
  2. GitHub: issue #17, 🤠 Review Queue
  3. GitHub: PR #3
  4. GitHub: PR #14
  5. GitHub: PR #10
  6. Slack: `#daily-pr-review`, plus one DM
  7. `agent/README.md` (agent table) or `diagrams/workflow.png`
  8. Trace **A**, **B**, **C** (links at the bottom)
  9. LangSmith: `pr-wrangler-checks` dataset, then the experiment
  10. VS Code: `scripts/check_guide.py`, `middleware/stash_results.py`, `tools/queue_facts.py`, `middleware/tool_scope.py`, `tools/mcp.py`, `tools/publish.py`

0:00 Problem

On screen	Say
GitHub: 15 open PRs	"Picture a team of seven developers, all shipping with AI coding tools. Writing code got cheap. Reviewing it didn't. Every morning the tech lead opens GitHub to fifteen pull requests. Some depend on each other, some touch the same lines, some quietly add risky dependencies. Where do you even start? That's the problem PR Wrangler solves."

0:30 Demo

On screen	Say
Issue #17, Review Queue	"This is what PR Wrangler produced on its last run. A full run reviews all fifteen PRs and takes about eleven minutes on Groq's free tier, so I'll show you the result. A pinned Review Queue: every PR in order, with a reason. Quick wins first, PRs that should be reviewed together, and the ones that are blocked."
PR list, labels visible	"Every PR is labeled with its place in the queue, its lane and its risk."
PR #3: brief, "Start here"	"Every PR gets a brief. This one adds a 'Clear completed' button. Looks harmless, but the agent found the bug: clicking it deletes your active to-dos and keeps the finished ones. It points to the exact line to start reading, marks it high risk, and sends it back to its author first."
PR #14	"The dependency gate checks new packages. This PR pulls in an old lodash with known vulnerabilities, just for one debounce function. And it edits the same lines as PR six, so it says merge six first."
PR #10	"And this one was built on PR two's branch, so it's blocked until two merges."
Slack digest, then one DM	"The team gets a Slack digest, and each author whose PR needs a fix first gets a DM. It never approves or merges anything. It writes the first draft of the review, and a human decides."

1:30 Architecture

On screen	Say
README agent table or `diagrams/workflow.png`	"It's built on LangSmith Managed Deep Agents. That gives me the harness: the agent loop, subagents, a file system the agents share, middleware hooks, and tracing. I only write the parts that are specific to PR review."
Same	"One orchestrator and three subagents, each on its own open model through Groq: a review guide for each PR, a dependency gate, and a queue planner. Each one returns a typed result, not free text."
VS Code: `tools/mcp.py`	"GitHub comes in through the GitHub MCP server: read-only tools, in lockdown mode, because PR text is written by people and the agent shouldn't take orders from it."
VS Code: `tools/` folder	"The facts come from plain Python tools: which packages changed, which PRs edit the same lines, and one publish tool that writes everything back to GitHub. Slack goes out through the Slack API as the PR Wrangler bot."
VS Code: `schedules/morning_queue.py`, then `channels/github.py`	"Two triggers: a schedule every weekday at nine, and a GitHub webhook, so when one PR changes, only that PR is re-reviewed. They're written; I run it locally because deploying needs a paid LangSmith plan."

On screen	Say
Trace **A**	"Here's a full run in the trace: the orchestrator hands each PR to a fresh review-guide subagent, then the dependency gate, then the queue planner."
VS Code: `middleware/stash_results.py`	"And context engineering: fifteen full briefs would flood the orchestrator, so each result is saved as a file and the orchestrator only gets a one-line summary."

2:40 Guardrails

Guardrail	On screen	Say
Each agent only sees its own tools	Trace **B**: Tools on a review-guide call (then the queue-planner call)	"Each agent only sees its own tools. The review guide can only read a pull request. The queue planner gets no tools at all."
Work happens in order	Trace **C**: the refusal	"Here's a guard firing for real. The orchestrator tried to plan before every PR was reviewed. The guard refused, listed the PRs still left, and it went back and finished them."
Ungrounded findings get dropped	VS Code: `stash_results.py`, `_fix_already_in_code` (3 sec)	"If the model suggests a 'fix' that's already exactly what the code says, that finding is dropped."
Read-only, never merges	VS Code: `tools/publish.py` (3 sec), then a PR comment footer	"GitHub access is read-only. The only write is one publish tool that never duplicates a comment, and it never approves or merges. A human decides."

3:20 Evals in LangSmith

On screen	Say
Experiment table, all 24 rows	"These are the evals. When I built the fifteen test PRs, I planted the right answers: a bug in PR three, a vulnerable package in fourteen, two pairs of PRs that edit the same lines, one PR that's blocked. I turned each planted answer into a check. That's these twenty-four rows."
Click row `#3 finds the inverted filter as a bug` → side panel	"Take this one. The input is the check and the subagent it tests: the review guide. The reference output is the answer I planted: 'clear completed' keeps the done to-dos and deletes the active ones. And the output is what the agent actually wrote for PR three: it found it, as a bug, on the filter line."
VS Code: `scripts/check_guide.py`, the `3:` block	"Pass or fail isn't another model's opinion. Each check is a few lines of code. This one passes if the brief has a finding marked 'bug' that mentions the filter. The answers are known, so plain code judges them: free, and the same result every time."
Back to the table, the `passed` column	"All twenty-four pass. And whenever I change a prompt or a model, I run a new experiment against the same dataset and compare, so I know the change didn't break anything."

4:05 Close

On screen	Say
Repo	"Next: run it on every PR change through the webhook I've written, and score every prompt change with that experiment. Code's on GitHub."

LangSmith links

	What	Link
A	Full run (Oct 17, 11 min): orchestrator delegating to all three subagents	open
B	Same run, a review-guide model call. Tools: only `github_pull_request_read` + `ReviewBrief`	open
	(contrast) a queue-planner call: no tools	open
C	Same run, 3 min in: `AllBriefsBeforePlanning` refuses to plan early	open
	Experiment, 24/24	open

Recording rules

- One take. Mess up a line, pause, say it again, cut later.
- Screen and voice only.
- Good enough to send a friend = done.